

Le Lean Software Development

Compléter Agile en optimisant les flux

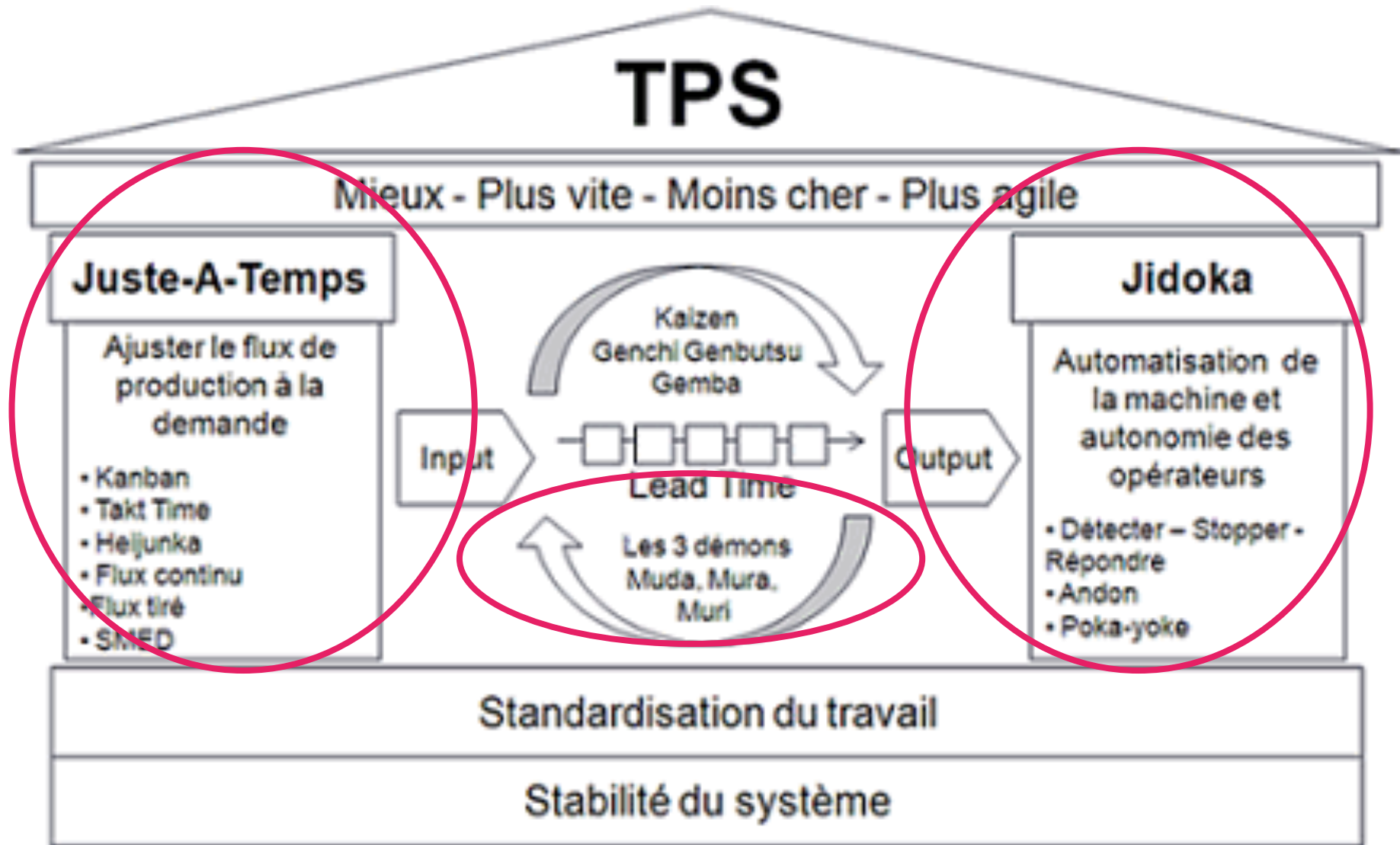
01 – Naissance du Lean

De TPS au Lean Software Development

01.01 TPS

Jidoka et Just-At-Time

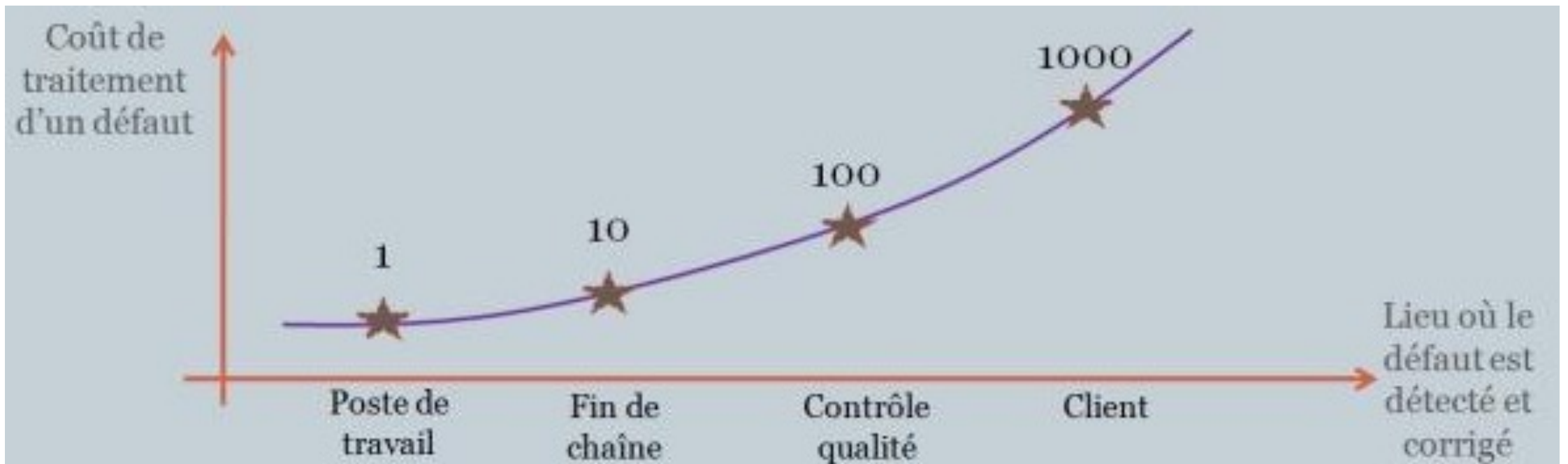
Toyota Production System (TPS)



Jidoka

Affirmation

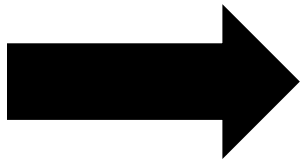
Régler les problèmes au plus tôt



Jidoka

Comment le mettre en œuvre ?

- Andon : arrêter automatiquement la production lorsqu'un problème est détecté
- Separation of man & machine : au lieu qu'un opérateur regarde une machine pour vérifier chaque pièce, la machine détecte elle-même une pièce défectueuse, s'arrête et alerte l'opérateur avec un **Andon**



Affirmation

Jidoka signifie *automatisation avec une touche d'humain*

- machine = production et détection
- humain = analyse et résolution du problème.

- Poka-Yoke (error proofing) : concevoir le processus pour empêcher une erreur ou la détecter immédiatement avant qu'elle ne produise un défaut.
 - Exemple : une pièce ne peut être insérée que dans le bon sens grâce à sa forme → impossible de faire l'erreur.

Jidoka

L'humain reste donc très important

- Toyota a pris le pari contraire de virer des machines pour remettre des humains sur les chaines de montage.
 - là où effectivement les machines permettront des gains à court terme en éliminant les « wastes » que sont les pertes d'attention, les pauses café, etc... les humains, eux, apprennent.
 - On a donc un plafond de connaissance dans un cas qui sera difficilement rattrapable face à une concurrence ayant gardé des personnes derrière les machines.

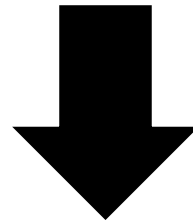
« Machines are great at repetitive tasks but people are outstanding problem-solvers »

Just-in-time

Affirmation

Faire ce qui est nécessaire, seulement lorsque c'est nécessaire, et seulement dans la quantité qui est nécessaire.

- ne rien produire qui ne soit immédiatement utile.



On réduit le Stock et le Gaspillage

Just-in-time

Comment le mettre en œuvre : Pull System



- Les produits sont « poussés » vers le marché
- **Les produits sont « tirés » par le marché** = on produit en fonction des commandes réelles des clients

01.02 Lutte contre les trois démons

Muda, Mura, Muri

Luttes contre les gaspillages et de recherche de la perfection du processus

- Le Muda : le démon du gaspillage (7 sources de gaspillage)
- Le Mura : le démon de l'irrégularité
- Le Muri : le démon de l'excès

Les 3M : MUDA, MURA, MURI

MUDA
(Gaspillage)



MURA
(Irrégularité)



MURI
(Surcharge)



Réduire le gaspillage (Muda)

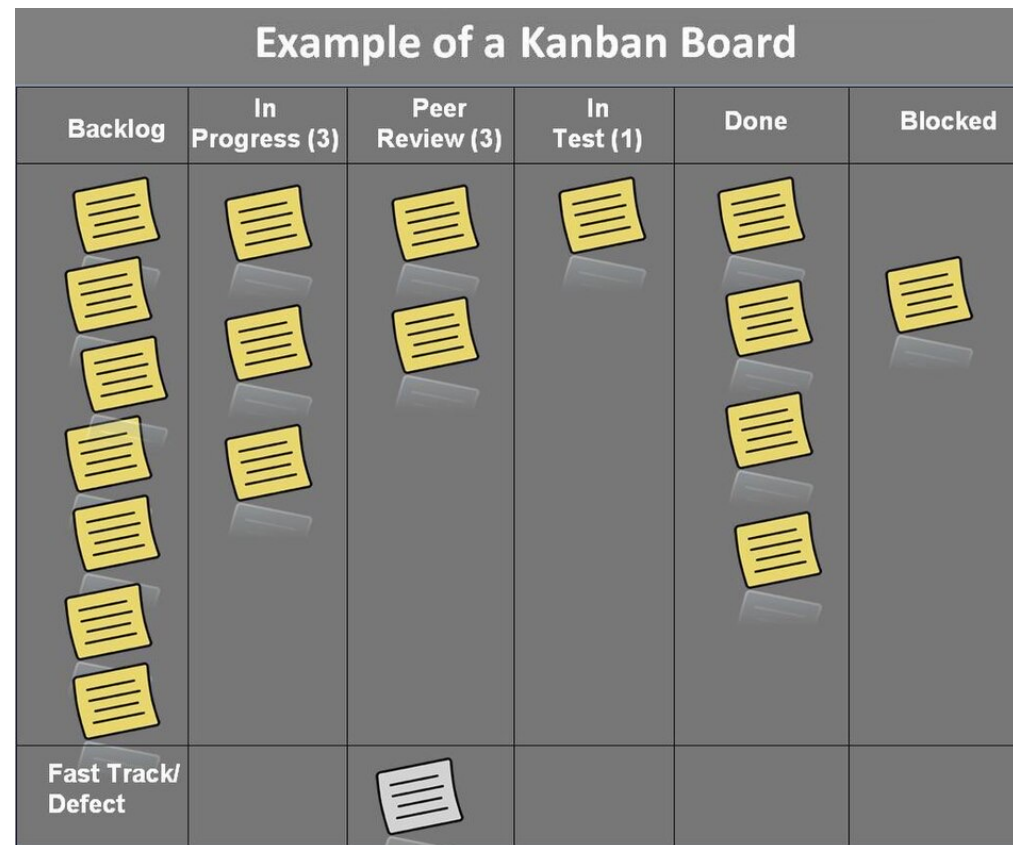
GASPILLAGE (MUDA)	EXEMPLE EN PRODUCTION	EXEMPLE DANS LES SERVICES
Génération de rebus	Un perçage mal positionné sur une pièce usinée	Une chambre d'hôtel attribuée avec 1 lit double quand le client a réservé une chambre avec 2 lits simples
Attente	Un opérateur attend la pièce en train d'être usinée au poste de travail précédent pour la nettoyer ensuite	Un médecin qui attend son prochain patient qui est en retard
Stockage	Un stock de 200 meubles qui sont en attente de mise en peinture	Des patients dans la salle d'attente d'un médecin
Déplacements	Un opérateur va chercher des outils de fabrication dans un atelier situé à 100 mètres de son poste de production	Un assistant de ventes qui doit aller vérifier dans l'entrepôt si l'article commandé par un client est bien disponible en stock
Transports	Un verre fabriqué pour contenir de la moutarde qui est transporté dans une autre usine pour être décoré	Un dossier qui est transféré dans un autre bâtiment pour être signé par un responsable qui y a son bureau
Process inutiles	Un verre fabriqué avec l'ajout d'une décoration qui n'a pas été demandée par le client	Un responsable de service qui veut relire chaque réponse qui est envoyée à un client
Surproduction	Produire 820 pièces quand le client en a commandées 800	Un hôtel qui réserve deux chambres pour un client unique (rare, mais déjà vu)

01.03 Quelques outils

Tableau Kanban et Value Stream Map

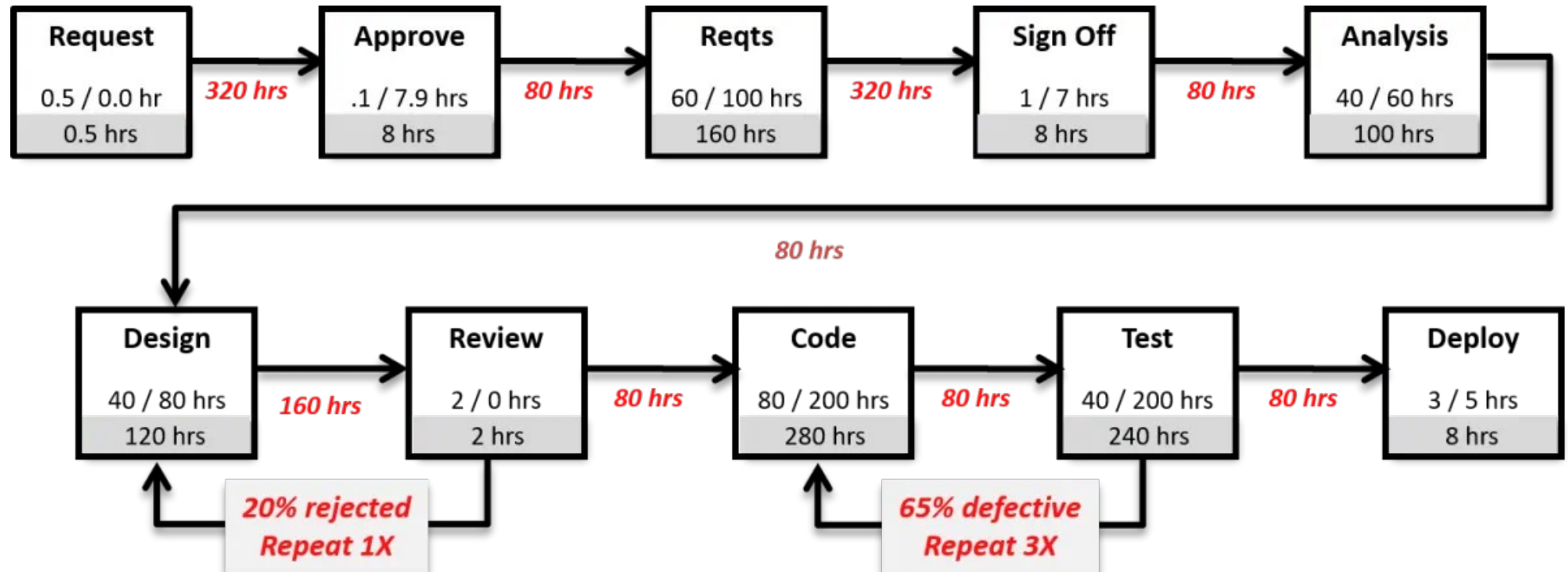
JAT via Kanban

- Tableau Kanban : permet d'avoir une vue globale des différentes étapes de fabrication



Value Stream Map pour améliorer les flux

- Value Stream Map : représenter l'ensemble du flux de production afin d'identifier les **étapes qui créent de la valeur** et **celles qui génèrent du gaspillage**.



02. Le Lean Software Dev.

Le mouvement Lean appliqué au logiciel

Objectifs du Lean Software Development

Objectif du Lean Software Development

Obtenir pour l'activité du développement logiciel des résultats équivalents à ceux obtenus par les diverses applications du lean (production industrielle, services, ingénierie, santé).

Le Lean est

- Une approche
- Qui vise la satisfaction client
- Qui vise la rentabilité par l'élimination du gaspillage
- Dans un environnement imprévisible
- Via la résolution de problème

Lean Software Development: 7 principes fondateurs

<https://www.qualitystreet.fr/2008/08/27/lean-software-development-7-principes-fondateurs/>



GUIDED TOUR

This book contains seven chapters devoted to seven lean principles and thinking tools for translating each principle into agile practices. A brief introduction to the seven lean principles concludes this introduction.

1. **Eliminate waste.** Waste is anything that does not add value to a product, value as perceived by the customer. In lean thinking, the concept of waste is a high hurdle. If a component is sitting on a shelf gathering dust, that is waste. If a development cycle has collected requirements in a book gathering dust, that is waste. If a manufacturing plant makes more stuff than is immediately needed, that is waste. If developers code more features than are immediately needed, that is waste. In manufacturing, moving product around is waste. In product development, handing off development from one group to another is waste. The ideal is to find out what a customer wants, and then make or develop it and deliver exactly what they want, virtually immediately. Whatever gets in the way of rapidly satisfying a customer need is waste.
2. **Amplify learning.** Development is an exercise in discovery, while production is an exercise in reducing variation, and for this reason, a lean approach to development results in practices that are quite different than lean production practices. Development is like creating a recipe, while production is like making the dish. Recipes are designed by experienced chefs who have developed an instinct for what works and the capability to adapt available ingredients to suit the occasion. Yet even great chefs produce several variations of a new dish as they iterate toward a recipe that will taste great and be easy to reproduce. Chefs are not expected to get a recipe perfect on the first attempt; they are expected to produce several variations on a theme as part of the learning process.¹² Software development is best conceived of as a similar learning process with the added challenge that development teams are large and the results are far more complex than a recipe. The best approach to improving a software development environment is to amplify learning.
3. **Decide as late as possible.** Development practices that provide for late decision making are effective in domains that involve uncertainty, because they provide an options-based approach. In the face of uncertainty, most economic markets develop options to provide a way for investors to avoid locking in decisions until the future is closer and easier to predict. Delaying decisions is valuable because better decisions can be made when they are based on fact, not speculation. In an evolving market, keeping design options open is more valuable than committing early. A key strategy for delaying commitments when developing a complex system is to build a capacity for change into the system.
4. **Deliver as fast as possible.** Until recently, rapid software development has not been valued; taking a careful, don't-make-any-mistakes approach has seemed to be more important. But it is time for "speed costs more" to join "quality costs more" on the list of debunked myths.¹³ Rapid development has many advantages. Without speed, you cannot delay decisions. Without speed, you do not have reliable feedback. In development the discovery cycle is critical for learning: Design, implement, feedback, improve. The shorter these cycles are, the more can be learned. Speed assures that customers get what they need now, not what they needed yesterday. It also allows them to delay making up their minds about what they really want until they know more. Compressing the value stream as much as possible is a fundamental lean strategy for eliminating waste.

5. **Empower the team.** Top-notch execution lies in getting the details right, and no one understands the details better than the people who actually do the work. Involving developers in the details of technical decisions is fundamental to achieving excellence. The people on the front line combine the knowledge of the minute details with the power of many minds. When equipped with necessary expertise and guided by a leader, they will make better technical decisions and better process decisions than anyone can make for them. Because decisions are made late and execution is fast, it is not possible for a central authority to orchestrate activities of workers. Thus, lean practices use pull techniques to schedule work and contain local signaling mechanisms so workers can let each other know what needs to be done. In lean software development, the pull mechanism is an agreement to deliver increasingly refined versions of working software at regular intervals. Local signaling occurs through visible charts, daily meetings, frequent integration, and comprehensive testing.

6. **Build integrity in.** A system is perceived to have integrity when a user thinks, “Yes! That is exactly what I want. Somebody got inside my mind!” Market share is a rough measure of perceived integrity for products, because it measures customer perception over time.¹⁴ Conceptual integrity means that the system’s central concepts work together as a smooth, cohesive whole, and it is a critical factor in creating perceived integrity.¹⁵ Software needs an additional level of integrity—it must maintain its usefulness over time. Software is usually expected to evolve gracefully as it adapts to the future. Software with integrity has a coherent architecture, scores high on usability and fitness for purpose, and is maintainable, adaptable, and extensible. Research has shown that integrity comes from wise leadership, relevant expertise, effective communication, and healthy discipline; processes, procedures, and measurements are not adequate substitutes.

7. **See the whole.** Integrity in complex systems requires a deep expertise in many diverse areas. One of the most intractable problems with product development is that experts in any area (e.g., database or GUI) have a tendency to maximize the performance of the part of the product representing their own specialty rather than focusing on overall system performance. Quite often, the common good suffers if people attend first to their own specialized interests. When individuals or organizations are measured on their specialized contribution rather than overall performance, suboptimization is likely to result. This problem is even more pronounced when two organizations contract with each other, because people will naturally want to maximize the performance of their own company. It is challenging to implement practices that avoid suboptimization in a large organization, and it is an order of magnitude more difficult when contracts are involved.

Lean Software Development: 7 principes fondateurs

Résumé en une phrase

1. **Éliminer le gaspillage** : Supprimer tout ce qui ne crée pas directement de valeur pour le client.
2. **Amplifier l'apprentissage** : Favoriser des cycles courts d'expérimentation et de feedback pour apprendre et s'améliorer continuellement.
3. **Décider le plus tard possible** : Reporter les décisions irréversibles jusqu'à disposer de suffisamment d'informations fiables.
4. **Livrer le plus vite possible** : Réduire le temps entre conception, livraison et feedback pour accélérer l'apprentissage et la création de valeur.
5. **Responsabiliser l'équipe** : Donner aux personnes qui réalisent le travail l'autonomie nécessaire pour prendre les meilleures décisions.
6. **Intégrer la qualité dès la conception** : Concevoir un produit cohérent, utile, maintenable et capable d'évoluer durablement.
7. **Voir le système dans son ensemble** : Optimiser la performance globale plutôt que chaque composant, équipe ou spécialité séparément.

02.01 Réduire le gaspillage

Les principales sources de gaspillage en dev. logiciel

Réduire le gaspillage

Question :

Quelles sont les principales sources de gaspillage ?

« Les 3 principales sources de gaspillage sont la *non-qualité*, le *temps d'attente* et les *stocks* »



Réduire le gaspillage

Les 7 sources de gaspillage Lean et Lean Software Development

Table 1.1 *The Seven Wastes*

The Seven Wastes of Manufacturing	The Seven Wastes of Software Development
Inventory	Partially Done Work
Extra Processing	Extra Processes
Overproduction	Extra Features
Transportation	Task Switching
Waiting	Waiting
Motion	Motion
Defects	Defects

Partially Done Work

Partially done software development has a tendency to become obsolete, and it gets in the way of other development that might need to be done. But the big problem with partially done software is that you might have no idea whether or not it will eventually work. Sure, you have a stack of requirements and design documents. You may even have a pile of code, which may even be unit tested. But until the software is integrated into the rest of the environment, you don't really know what problems might be lurking, and until the software is actually in production, you don't really know if it will solve the business problem.

Partially done development ties up resources in investments that have yet to yield results. In software development these investments are sometimes capitalized, and depreciation starts when the software goes into production. What if the system never makes it into production? Then there is a big investment to write off. Partially done software development can carry huge financial risks. Minimizing partially done software development is a risk-reduction as well as a waste-reduction strategy.

Extra Processes

Do you ever ask, Is all that paperwork really necessary? Paperwork consumes resources. Paperwork slows down response time. Paperwork hides quality problems. Paperwork gets lost. Paperwork degrades and becomes obsolete. Paperwork that no one cares to read adds no value.

Many software development processes require paperwork for customer sign-off, or to provide traceability, or to get approval for a change. Does your customer really find this makes the product more valuable to them? Just because paperwork is a required deliverable does not mean that it adds value. If you must produce paperwork that adds little customer value, there are three rules to remember: Keep it short. Keep it high level. Do it off line.

Safety-critical systems are frequently regulated and are often required to have written requirements, traceable to code. In this case, formatting the requirements so they can be easily evaluated and checked for completeness may qualify as a value-adding activity. Look for a table-driven or template-driven format that reduces the requirements to a condensed format that both users and developers can rapidly understand and validate.

A good test of the value of paperwork is to see if there is someone waiting for what is being produced. If an analyst fills out templates, makes tables, or writes use cases that others are eager to use—for coding, testing, and writing training manuals—then these probably add value. Even so, there should be a constant search for the most efficient, effective means to transmit the information. Consider writing customer tests

instead of requirements. In general, delay documenting the details of desired features until the iteration in which they are implemented.

Extra Features

It may seem like a good idea to put some extra features into a system just in case they are needed. Developers might like to add a new technical capability just to see how it works. This may seem harmless, but on the contrary, it is serious waste. Every bit of code in the system has to be tracked, compiled, integrated, and tested every time the code is touched, and then it has to be maintained for the life of the system. Every bit of code increases complexity and is a potential failure point. There is a great possibility that extra code will become obsolete before it's used; after all, there wasn't any real call for it in the first place. If code is not needed *now*, putting it into the system is a waste. Resist the temptation.

Task Switching

Assigning people to multiple projects is a source of waste. Every time software developers switch between tasks, a significant switching time is incurred as they get their thoughts gathered and get into the flow of the new task.⁵ Belonging to multiple teams usually causes more interruptions and thus more task switching. This task switching time is waste.

The fastest way to complete two projects that use the same resources is to do them one at a time. Say you have two projects that should each take two weeks. If you start one of them, it should be done in two weeks. When it's done, you can start the second project, and it should be done in two weeks. What if you start both projects together and expect people to switch between them? First of all, neither one will be done in two weeks, but will they both be done in four weeks? When you add the switching time, they will probably take closer to five weeks.⁶

It is difficult to resist the temptation to start several projects at the same time, but releasing too much work into a software development organization creates a lot of waste, since it actually slows things down. Work moves much faster through a pipeline that is not filled to capacity, as we discuss in the section on queueing theory in Chapter 4, "Deliver as Fast as Possible."

Waiting

One of the biggest wastes in software development is usually waiting for things to happen. Delays in starting a project, delays in staffing, delays due to excessive requirements documentation, delays in reviews and approvals, delays in testing, and delays in deployment are waste. Delays are common in most software development processes, and it seems counterintuitive to think of these delays as a waste. It would seem that at worst, delays are neutral.

So what's wrong with waiting? Delay keeps the customer from realizing value as quickly as possible. When a critical customer need arrives in your development organization, the speed with which you can respond is directly related to the systemic delays in your development cycle.

For some environments, delay may not loom as large as other problems. However, if you are developing software for an evolving domain, delays in development are more serious. A fundamental lean principle is to delay decisions until the last possible moment so you can make the most informed decision possible. This is an options-based approach to software development, and it is the best way to deal with uncertainty, as we discuss in Chapter 3, “Decide as Late as Possible.” You cannot delay decisions, however, if you cannot implement rapidly once a decision is made.

Motion

When a developer has a question, how much motion does it take to find out the answer? Are people at hand to help with a technical problem? Is the customer or customer representative readily accessible to answer a question about features? Can the developer find out the results of tests without walking down the hall? Development is an activity that requires great concentration, so walking down the hall takes a lot more time than you might think. It will probably take the developer several times as long to reestablish focus as it took to get the question answered. It is for this reason that agile software development practices generally recommend that a team work in a single workroom where everyone has access to developers, to testers, and to customers or customer representatives.

People aren't the only things that move—various artifacts move also. Requirements may move from analysts to designers, and then design documents move from designers to programmers, and then code moves from coders to testers, and so on. Each handoff of an artifact is fraught with opportunities for waste. The biggest waste of all in document handoffs is that documents don't—can't, really—contain all of the information that the next person in line needs to know. Great amounts of tacit knowledge remain with the creator of the document and never get handed off to the re-

ceiver. Moving artifacts from one group to another is a huge source of waste in software development.

Defects

The amount of waste caused by a defect is the product of the defect impact and the time it goes undetected. A critical defect that is detected in three minutes is not a big source of waste. A minor defect that is not discovered for weeks is a much bigger waste. The way to reduce the impact of defects is to find them as soon as they occur. Thus, the way to reduce the waste due to defects is to test immediately, integrate often, and release to production as soon as possible.

Comment découvrir le gaspillage ?

« Map your Value Stream »

In the book *Lean Thinking*,⁷ James Womack and Daniel Jones chronicle the journey of a cola can from raw material to consumption. It looks like Figure 1.1.

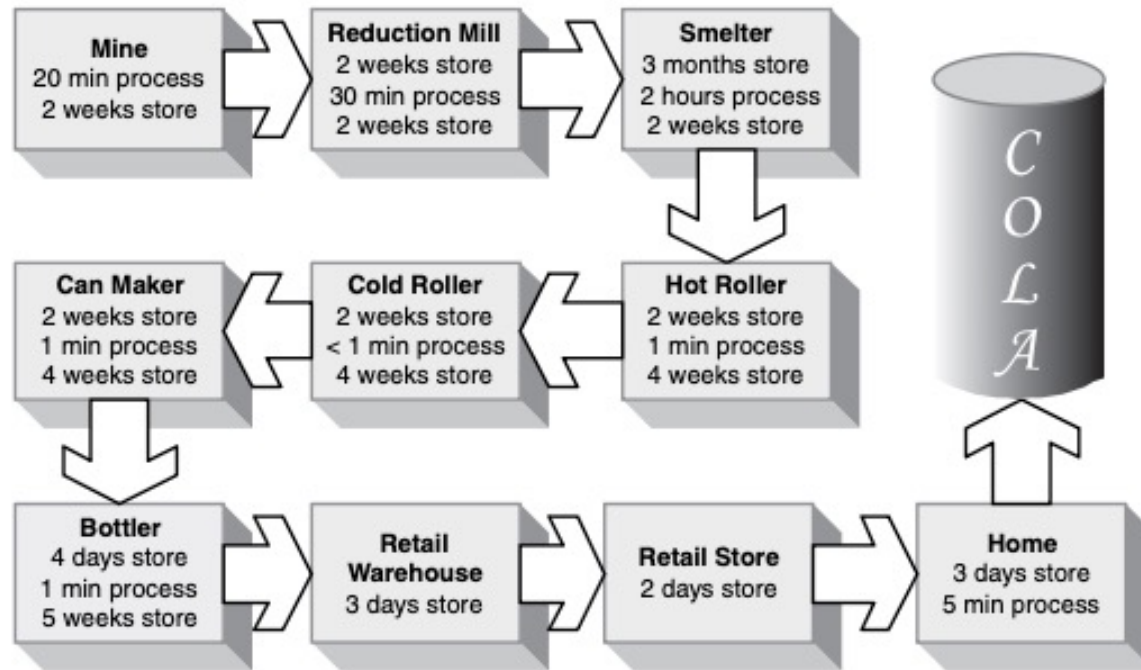


Figure 1.1 Value stream for cola cans.

Comment découvrir le gaspillage ?

Value Stream Map traditionnelle vs Agile

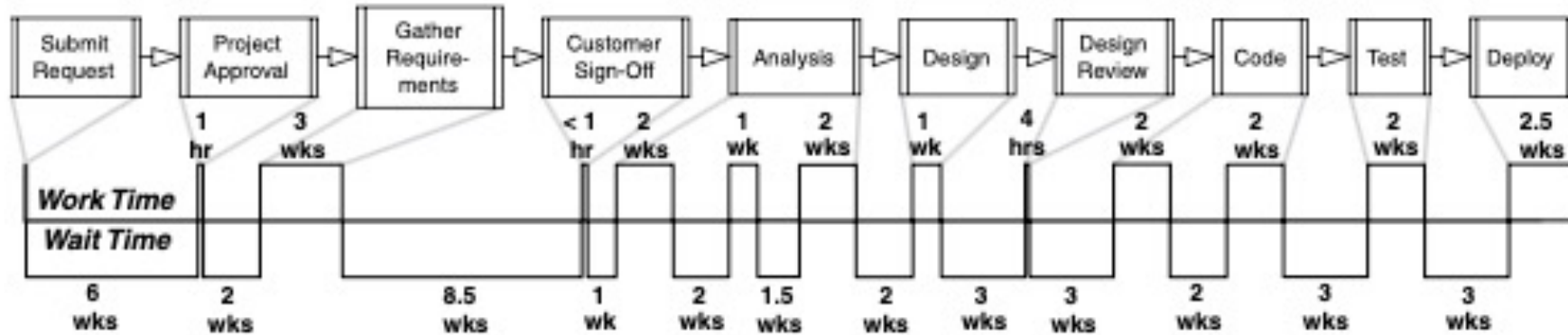


Figure 1.2 Traditional value stream map.

- ≈ 1 an entre la demande et la mise en production.
- Seulement $\approx 1/3$ du temps crée réellement de la valeur.
- L'essentiel du délai vient des temps d'attente entre les étapes (validation, ressources disponibles, revues...).
- **Le feedback client arrive très tard, ce qui rend les changements risqués et coûteux.**

Comment découvrir le gaspillage ?

Value Stream Map traditionnelle vs Agile

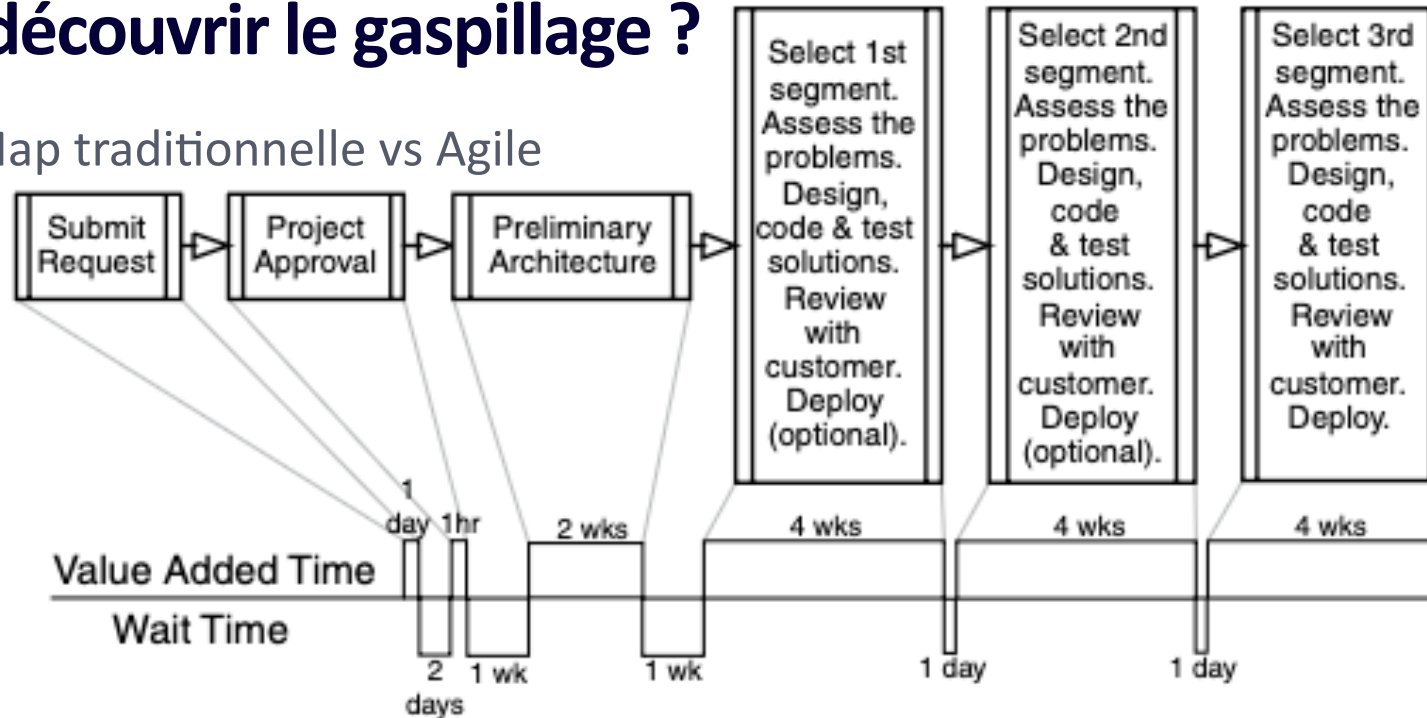


Figure 1.3 Agile value stream map.

- Besoins définis au fil du développement
- Design + développement intégrés → suppression des longues revues intermédiaires.
- Feedback client continu → plutôt qu'une validation unique en fin de projet.
- Déploiement régulier → livraison plus rapide. Résultat :
- **≈ 3 mois au lieu d'≈ 1 an**

02.02 Les autres principes, exemple

Créer de la connaissance, retarder engagement, livrer rapidement

Créer des connaissances

« Le meilleur moyen de gérer des projet de développement logiciel difficile est d'augmenter la quantité de feedback »



Retarder l'engagement

« Eviter de prendre une décision trop tôt ou quand le futur est incertain. »



Livrer rapidement

Question :

Qu'est-ce que cela implique ?

« Plus les itérations sont courtes, plus le feedback est rapide, plus nous apprenons et incorporons les remarques dans la prochaine itération »

